EXPERIMENT NO: 5

Evaluation of Database Credential Strength and Password Hash Security Using Hashcat

AIM:

To evaluate the strength of database credentials and demonstrate password hash security using MySQL 8.0.46 and Hashcat 7.1.2 on a Windows system.

Objectives

1. To configure and examine MySQL database user credentials.
2. To evaluate password strength using MySQL's validate_password component.
3. To verify password complexity requirements such as length, uppercase/lowercase characters, numbers, and special characters.
4. To examine MySQL user authentication plugins and account status.
5. To verify database user privileges and demonstrate the principle of least privilege.
6. To generate a controlled SHA-256 password hash for testing.
7. To use Hashcat to perform a dictionary-based password recovery attack against the test hash.
8. To understand the importance of strong passwords and secure password-hashing mechanisms.

Requirements

Hardware	Software
Windows 10/11 Computer/Laptop	MySQL Community Server 8.0.46
Minimum 4 GB RAM	MySQL Command-Line Client / Workbench
Dual-Core Processor or better	Hashcat 7.1.2
AMD Radeon / OpenCL-compatible GPU	PowerShell / Command Prompt
Minimum 2 GB Free Storage	Notepad / Text Editor

Theory

Database security depends heavily on the strength of user credentials and the security of authentication mechanisms. Weak passwords such as common words, short passwords, or predictable combinations can be vulnerable to dictionary and brute-force attacks. A database should therefore enforce password complexity requirements and restrict user privileges according to the principle of least privilege.

MySQL provides the validate_password component to enforce password-strength policies. In this practical, the validate_password component was installed and configured with a MEDIUM policy. The configured policy required a minimum password length of 8 characters, at least one mixed-case character, one number, and one special character. It also checked whether the password contained the username.

MySQL 8.0 uses the caching_sha2_password authentication plugin by default for newly created accounts. The authentication plugin determines how the server authenticates users and protects authentication credentials.

Password hashing converts a password into a fixed-length value using a cryptographic hash function. Hash functions are designed to be one-way, meaning the original password should not be directly obtained from the hash. However, weak passwords can still be recovered through dictionary or brute-force attacks because an attacker can hash candidate passwords and compare the resulting hashes.

Hashcat is a password-recovery and security-auditing tool that supports many hash algorithms. In this experiment, a controlled SHA-256 hash of the test password password123 was generated. Hashcat was then supplied with a small dictionary containing possible passwords. It successfully recovered the test password, demonstrating that a weak password can be quickly identified when its hash is available and the password occurs in the attack dictionary.

Commands Used:

Command	Specification
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" --version	Checks the installed MySQL version.
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p	Connects to MySQL using the root account.
SELECT VERSION();	Displays the MySQL server version.
SELECT User, Host, plugin, account_locked, password_expired FROM mysql.user;	Examines database users, authentication plugins, account status, and password expiration.
INSTALL COMPONENT 'file://component_validate_password';	Installs the MySQL password-validation component.
SELECT * FROM mysql.component;	Verifies the installed password-validation component.
SHOW VARIABLES LIKE 'validate_password%';	Displays password-strength policy settings.
CREATE USER 'policy_test'@'localhost' IDENTIFIED BY 'abc';	Tests whether a weak password is rejected.
CREATE USER 'policy_test'@'localhost' IDENTIFIED BY 'M7!qZ#4vL@9xP2$k';	Tests whether a strong password satisfies the policy.

Command	Specification
SHOW GRANTS FOR 'policy_test'@'localhost';	Displays privileges assigned to the test user.
"password123" | Set-Content -NoNewline .\password.txt	Creates a controlled test password file.
$hash = (Get-FileHash .\password.txt -Algorithm SHA256).Hash	Generates the SHA-256 hash of the test password.
$hash | Set-Content -NoNewline .\test_hash.txt	Saves the generated hash to a file.
.\hashcat.exe --version	Displays the installed Hashcat version.
.\hashcat.exe -I	Displays available Hashcat computing devices.
.\hashcat.exe -m 1400 -a 0 .\test_hash.txt .\words.txt	Performs a SHA-256 dictionary attack using the test wordlist.
.\hashcat.exe -m 1400 .\test_hash.txt --show	Displays the successfully recovered password.

Procedure:

1. Install and configure MySQL Server 8.0.46 on Windows.
2. Open the MySQL Command-Line Client using the root account.
3. Display the existing database users and their authentication plugins.
4. Install the MySQL validate_password component.
5. Check the active password policy using SHOW VARIABLES LIKE 'validate_password%';.
6. Attempt to create a user using a weak password such as abc.
7. Observe that MySQL rejects the weak password because it does not satisfy the configured policy.
8. Create a test user using a strong password containing uppercase/lowercase characters, numbers, special characters, and sufficient length.
9. Verify that the strong password is accepted.
10. Check the authentication plugin, account status, and privileges of the test user.
11. Create a separate controlled test password, password123, for the hash-security demonstration.
12. Generate its SHA-256 hash using PowerShell.
13. Store the generated hash in test_hash.txt.
14. Create a small dictionary containing common candidate passwords.

15. Run Hashcat using SHA-256 mode (-m 1400) and dictionary attack mode (-a 0).
16. Observe that Hashcat successfully recovered password123 from the controlled test hash.
17. Analyze the results and determine the importance of strong passwords and secure authentication mechanisms.

Output:

1) MySQL Basic Installation:

```
C:\Users\CHAITANYA>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p
Enter password: *********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

2) Mysql operation:

```
mysql> SELECT User, Host, plugin, account_locked, password_expired
    -> FROM mysql.user;
+------------------+-----------+-----------------------+----------------+------------------+
| User             | Host      | plugin                | account_locked | password_expired |
+------------------+-----------+-----------------------+----------------+------------------+
| mysql.infoschema | localhost | caching_sha2_password | Y              | N                |
| mysql.session    | localhost | caching_sha2_password | Y              | N                |
| mysql.sys        | localhost | caching_sha2_password | Y              | N                |
| root             | localhost | caching_sha2_password | N              | N                |
+------------------+-----------+-----------------------+----------------+------------------+
4 rows in set (0.00 sec)

mysql> SELECT VERSION();
+-----------+
| VERSION() |
+-----------+
| 8.0.46    |
+-----------+
1 row in set (0.00 sec)
```

3) Creation of Database,Password and Allocations:

```
mysql> CREATE DATABASE dbaudit_practical;
Query OK, 1 row affected (1.12 sec)

mysql> USE dbaudit_practical;
Database changed
mysql> CREATE USER 'weak_test'@'localhost'
    -> IDENTIFIED BY 'Password123';
Query OK, 0 rows affected (0.64 sec)

mysql> CREATE USER 'strong_test'@'localhost'
    -> IDENTIFIED BY 'M7!qZ#4vL@9xP2$k';
Query OK, 0 rows affected (0.80 sec)

mysql> SELECT User, Host, plugin, account_locked, password_expired
    -> FROM mysql.user
    -> WHERE User IN ('weak_test','strong_test');
+-------------+-----------+-----------------------+----------------+------------------+
| User        | Host      | plugin                | account_locked | password_expired |
+-------------+-----------+-----------------------+----------------+------------------+
| strong_test | localhost | caching_sha2_password | N              | N                |
| weak_test   | localhost | caching_sha2_password | N              | N                |
+-------------+-----------+-----------------------+----------------+------------------+
2 rows in set (0.00 sec)
```

4) Showing Grants for weak and Strong ones:

```
mysql> SHOW GRANTS FOR 'weak_test'@'localhost';
+-------------------------------------------------+
| Grants for weak_test@localhost                  |
+-------------------------------------------------+
| GRANT USAGE ON *.* TO `weak_test`@`localhost` |
+-------------------------------------------------+
1 row in set (0.00 sec)

mysql> SHOW GRANTS FOR 'strong_test'@'localhost';
+-------------------------------------------------+
| Grants for strong_test@localhost                |
+-------------------------------------------------+
| GRANT USAGE ON *.* TO `strong_test`@`localhost` |
+-------------------------------------------------+
1 row in set (0.00 sec)
```

5) Displaying components and Variables of password:

```
mysql> SELECT * FROM mysql.component;
+--------------+--------------------+-----------------------------------+
| component_id | component_group_id | component_urn                     |
+--------------+--------------------+-----------------------------------+
|            1 |                  1 | file://component_validate_password |
+--------------+--------------------+-----------------------------------+
1 row in set (0.00 sec)

mysql> SHOW VARIABLES LIKE 'validate_password%';
+--------------------------------------------------+--------+
| Variable_name                                    | Value  |
+--------------------------------------------------+--------+
| validate_password.changed_characters_percentage | 0      |
| validate_password.check_user_name                | ON     |
| validate_password.dictionary_file                |        |
| validate_password.length                         | 8      |
| validate_password.mixed_case_count               | 1      |
| validate_password.number_count                   | 1      |
| validate_password.policy                         | MEDIUM |
| validate_password.special_char_count             | 1      |
+--------------------------------------------------+--------+
8 rows in set (0.01 sec)
```

6) Checking the policy requirements:

```
mysql> CREATE USER 'policy_test'@'localhost' IDENTIFIED BY 'abc';
ERROR 1819 (HY000): Your password does not satisfy the current policy requirements
mysql> CREATE USER 'policy_test'@'localhost'
    -> IDENTIFIED BY 'M7!qZ#4vL@9xP2$k';
Query OK, 0 rows affected (1.37 sec)
```

```
mysql> SELECT User, Host, plugin, account_locked, password_expired
    -> FROM mysql.user
    -> WHERE User = 'policy_test';
+-------------+-----------+-----------------------+----------------+------------------+
| User        | Host      | plugin                | account_locked | password_expired |
+-------------+-----------+-----------------------+----------------+------------------+
| policy_test | localhost | caching_sha2_password | N              | N                |
+-------------+-----------+-----------------------+----------------+------------------+
1 row in set (0.00 sec)

mysql> SHOW GRANTS FOR 'policy_test'@'localhost';
+-------------------------------------------------+
| Grants for policy_test@localhost                |
+-------------------------------------------------+
| GRANT USAGE ON *.* TO `policy_test`@`localhost` |
+-------------------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT User, Host, plugin, account_locked, password_expired
    -> FROM mysql.user;
+------------------+-----------+-----------------------+----------------+------------------+
| User             | Host      | plugin                | account_locked | password_expired |
+------------------+-----------+-----------------------+----------------+------------------+
| mysql.infoschema | localhost | caching_sha2_password | Y              | N                |
| mysql.session    | localhost | caching_sha2_password | Y              | N                |
| mysql.sys        | localhost | caching_sha2_password | Y              | N                |
| policy_test      | localhost | caching_sha2_password | N              | N                |
| root             | localhost | caching_sha2_password | N              | N                |
| strong_test      | localhost | caching_sha2_password | N              | N                |
| weak_test        | localhost | caching_sha2_password | N              | N                |
+------------------+-----------+-----------------------+----------------+------------------+
7 rows in set (0.00 sec)
```

7) Trying to create the user and validate:

```
mysql> CREATE USER 'weak_test'@'localhost'
    -> IDENTIFIED BY 'password';
ERROR 1819 (HY000): Your password does not satisfy the current policy requirements
mysql> SELECT User, Host, plugin, account_locked, password_expired
    -> FROM mysql.user
    -> WHERE User = 'policy_test';
+-------------+-----------+-----------------------+---------------+------------------+
| User        | Host      | plugin                | account_locked | password_expired |
+-------------+-----------+-----------------------+---------------+------------------+
| policy_test | localhost | caching_sha2_password | N              | N                |
+-------------+-----------+-----------------------+---------------+------------------+
1 row in set (0.00 sec)
```

8)Starting Powershell and Configuring hashcat:

```
PS C:\windows\System32> cd "C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2"
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> pwd

Path
----
C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2


PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> "password123" | Set-Content -NoNewline password.txt
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> Get-Content password.txt
password123
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> Get-FileHash password.txt -Algorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          EF92B778BAFE771E89245B89ECBC08A44A4E166C06659911881F383D4473E94F       C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2\password...
```

9)Creation of sample content:

```
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> notepad test_hash.txt
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> Get-Content test_hash.txt
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> @"
>> hello
>> admin
>> password
>> password123
>> welcome
>> qwerty
>> "@ | Set-Content words.txt
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> Get-Content words.txt
hello
admin
password
password123
welcome
qwerty
```

10)Getting the values directly.

```
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> $hash = (Get-FileHash .\password.txt -Algorithm SHA256).Hash
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> $hash | Set-Content -NoNewline .\test_hash.txt
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> (Get-Content .\test_hash.txt -Raw).Length
64
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> Get-Content .\test_hash.txt
EF92B778BAFE771E89245B89ECBC08A44A4E166C06659911881F383D4473E94F
```

11)Starting netcat and getting final view

```
PS C:\Users\CHAITANYA\Downloads\hashcat-7.1.2\hashcat-7.1.2> .\hashcat.exe -m 1400 -a 0 .\test_hash.txt .\words.txt
hashcat (v7.1.2) starting
```

```
The wordlist or mask that you are using is too small.     Finished autotune
This means that hashcat cannot use the full parallel power of your device(s).
Hashcat is expecting at least 65536 base words but only got 0.0% of that.
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f:password123

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1400 (SHA2-256)
Hash.Target......: ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881...73e94f
Time.Started.....: Wed Aug 26 11:17:38 2026 (0 secs)
Time.Estimated...: Wed Aug 26 11:17:38 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (.\words.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:     1501 H/s (0.09ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 6/6 (100.00%)
Rejected.........: 0/6 (0.00%)
Restore.Point....: 0/6 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: hello -> qwerty
Hardware.Mon.#01.: Temp:  0c Fan:  0% Util: 98% Core: 700MHz Mem: 800MHz Bus:16

Started: Wed Aug 26 11:17:01 2026
Stopped: Wed Aug 26 11:17:39 2026
```

Conclusion

The practical successfully evaluated database credential strength and password hash security using MySQL 8.0.46 and Hashcat 7.1.2. The MySQL validate_password component was enabled with a MEDIUM policy, which successfully rejected weak passwords and accepted passwords satisfying the required complexity rules. The database accounts were also examined for authentication method, account status, and privileges, with caching_sha2_password being used for authentication and the test account having only USAGE privileges. A controlled SHA-256 test hash was then generated and tested using Hashcat's dictionary attack mode. Hashcat successfully recovered the weak test password password123, demonstrating that weak passwords can be vulnerable to offline password-recovery attacks when the corresponding hash is available. Therefore, enforcing strong password policies, using secure authentication mechanisms, applying least-privilege access, and avoiding predictable passwords are essential practices for maintaining database security.